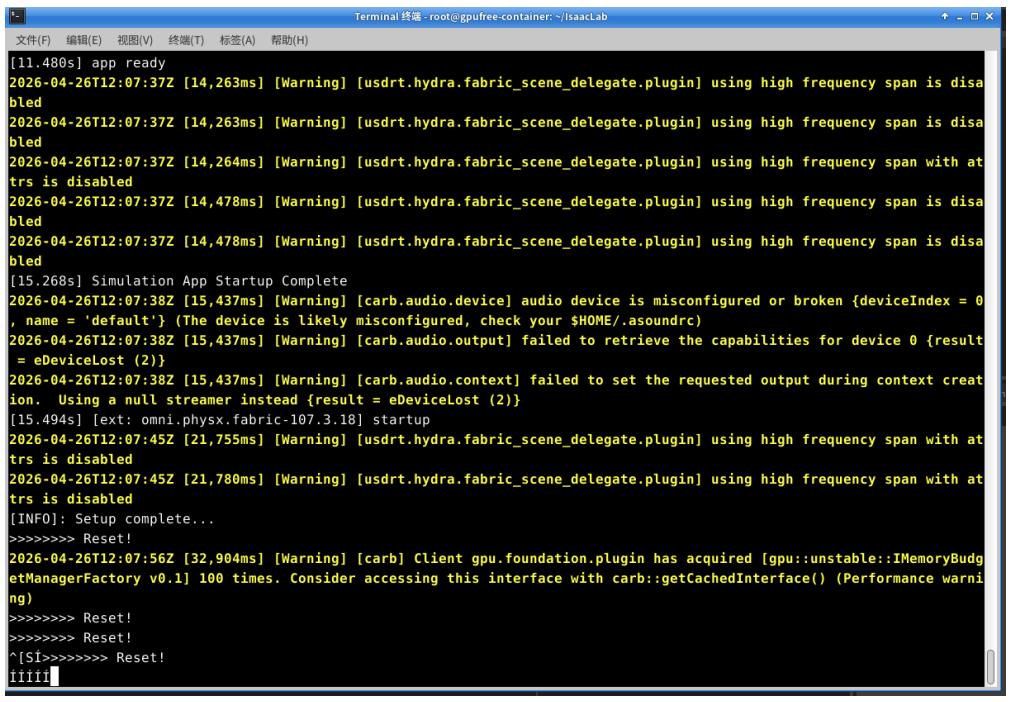
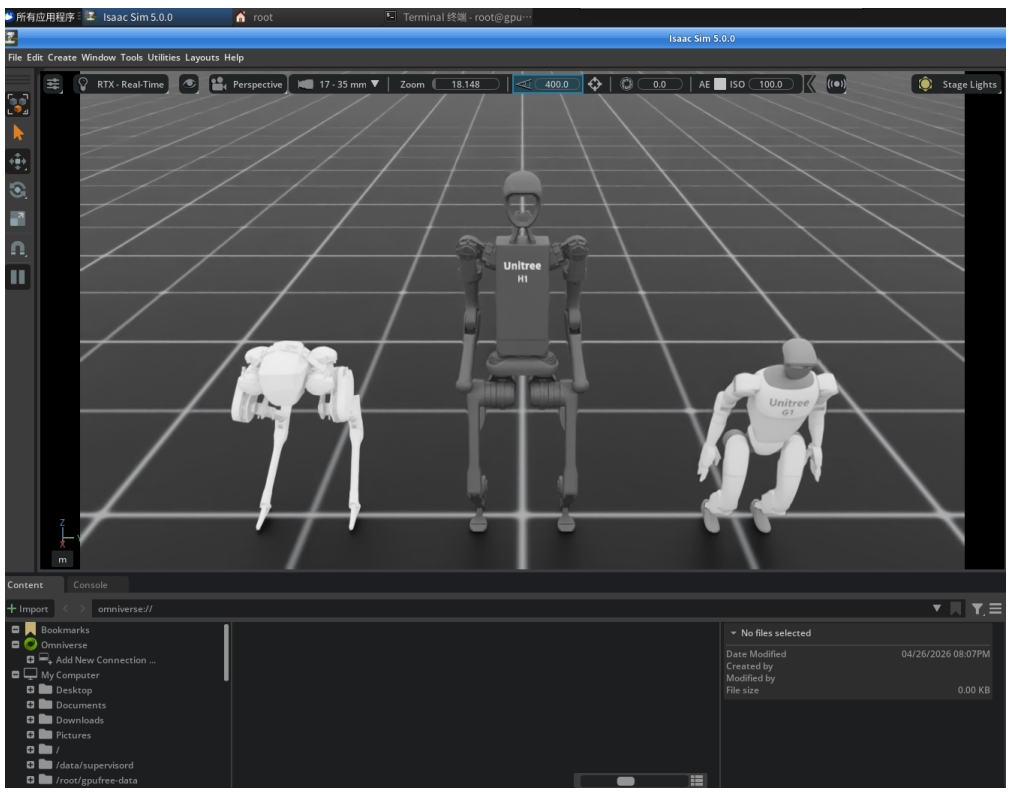
第八周作业-1

第八周作业 -1- 基础作业

1.1 环境与 Demo 验证

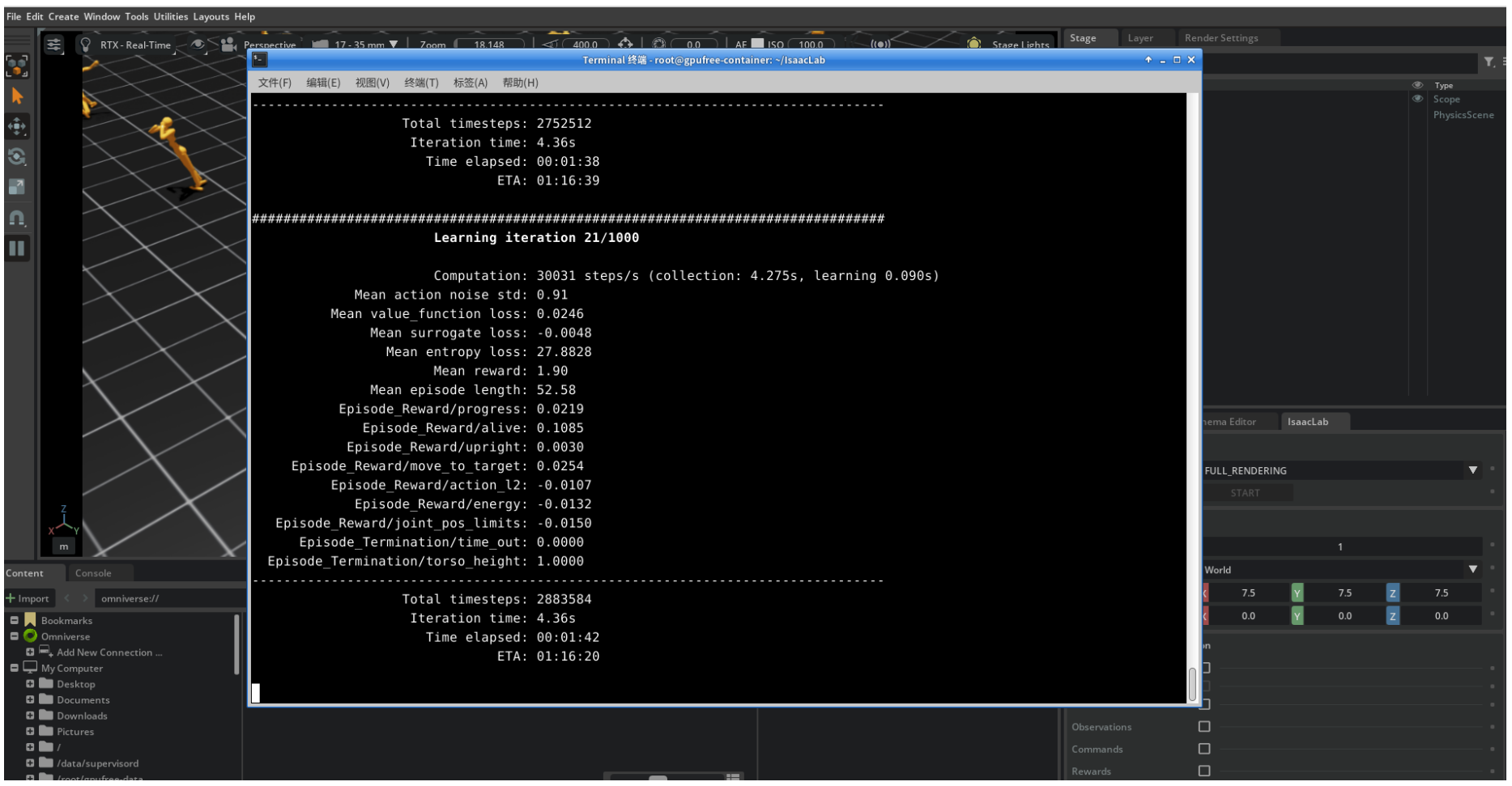
> 1. 配置 isaacsim + isaaclab 环境；
> 2. 运行一个双足机器人 Demo，确认 Isaac Lab 环境可正常启动：
> `./isaaclab.sh -p scripts/demos/bipeds.py`
> 3. 提供以下验证材料：
> - 终端截图 1 张：能看到 `(isaaclab)` 与成功启动命令；
> - Demo 运行截图 1 张：能看到双足机器人场景。

- IsaacLab 运行截图：
  - 
```
Terminal 终端 - root@gpufree-container: ~/IsaacLab
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
[11.480s] app ready
2026-04-26T12:07:37Z [14,263ms] [Warning] [usdrt.hydra.fabric_scene_delegate.plugin] using high frequency span is disa
bled
2026-04-26T12:07:37Z [14,263ms] [Warning] [usdrt.hydra.fabric_scene_delegate.plugin] using high frequency span is disa
bled
2026-04-26T12:07:37Z [14,264ms] [Warning] [usdrt.hydra.fabric_scene_delegate.plugin] using high frequency span with at
trs is disabled
2026-04-26T12:07:37Z [14,478ms] [Warning] [usdrt.hydra.fabric_scene_delegate.plugin] using high frequency span is disa
bled
2026-04-26T12:07:37Z [14,478ms] [Warning] [usdrt.hydra.fabric_scene_delegate.plugin] using high frequency span is disa
bled
[15.268s] Simulation App Startup Complete
2026-04-26T12:07:38Z [15,437ms] [Warning] [carb.audio.device] audio device is misconfigured or broken {deviceIndex = 0
, name = 'default'} (The device is likely misconfigured, check your $HOME/.asoundrc)
2026-04-26T12:07:38Z [15,437ms] [Warning] [carb.audio.output] failed to retrieve the capabilities for device 0 {result
 = eDeviceLost (2)}
2026-04-26T12:07:38Z [15,437ms] [Warning] [carb.audio.context] failed to set the requested output during context creat
ion.  Using a null streamer instead {result = eDeviceLost (2)}
[15.494s] [ext: omni.physx.fabric-107.3.18] startup
2026-04-26T12:07:45Z [21,755ms] [Warning] [usdrt.hydra.fabric_scene_delegate.plugin] using high frequency span with at
trs is disabled
2026-04-26T12:07:45Z [21,780ms] [Warning] [usdrt.hydra.fabric_scene_delegate.plugin] using high frequency span with at
trs is disabled
[INFO]: Setup complete...
>>>>>>>> Reset!
2026-04-26T12:07:56Z [32,904ms] [Warning] [carb] Client gpu.foundation.plugin has acquired [gpu::unstable::IMemoryBudg
etManagerFactory v0.1] 100 times. Consider accessing this interface with carb::getCachedInterface() (Performance warni
ng)
>>>>>>>> Reset!
>>>>>>>> Reset!
^[SÍ>>>>>>>> Reset!
ÍÍÍÍÍ
```
  - 

1.2 RL 训练

> 1. 启动 `Isaac-Humanoid-v0` 训练：
> `./isaaclab.sh -p scripts/reinforcement_learning/rsl_rl/train.py --task=Isaac-Humanoid-v0`
> 2. 观察训练成功启动后的终端输出，确认任务已经开始运行；
> 3. 训练运行一小段时间后，使用 `Ctrl + C` 结束训练；
> 4. 找到本次训练生成的 run 目录，例如：
> `logs/rsl_rl/humanoid/[run_name]/`
> 5. 回答以下问题：
> - `model_*.pt` 是什么？
> - `events.out.tfevents.*` 是什么？
> - `params/` 目录记录了什么？

1. 终端输出如下，任务运行正常

1. 

2. log 目录文件夹下的文件说明：
   1. `model_*.pt`：PyTorch 模型检查点（模型权重/状态）。作用：保存训练过程中（或训练后）的网络权重与可选训练状态（optimizer、step 等），用于恢复训练或做推理。
   2. `events.out.tfevents.*`:TensorBoard 事件文件.作用：TensorBoard 的日志文件，记录 scalars（loss/reward）、histograms、模型图像、训练步骤等可视化数据。
   3. `params/`：记录了训练时的配置和序列化参数

1.3 训练任务信息分析

> 结合训练启动输出、操作手册和课堂内容，回答以下问题：
>
> 1. `Isaac-Humanoid-v0` 的 Observation 是多少维？Action 是多少维？
> 2. 状态 `S_t` 中至少列出 5 类信息，并分别说明它们对行走决策有什么作用；
> 3. 为什么“上一帧动作”也会进入观测？
> 4. 奖励函数中至少列出 5 个 reward 项，并说明它们分别在奖励什么、惩罚什么；
> 5. 列出所有 termination 条件，并说明它们为什么重要；
> 6. 用一段话解释：为什么机器人 RL 训练需要并行仿真？
> 7. 用一段话解释：`Isaac Sim` 和 `Isaac Lab` 的分工分别是什么？

1. Observations 维度是 87 维，Action 维度为 21 维。

base_height	1
base_lin_vel	3
base_ang_vel	3
base_yaw_roll	2
base_angle_to_target	1
base_up_proj	1
base_heading_proj	1
joint_pos_norm	21
joint_vel_rel	21
feet_body_forces	12 (两个足，6D each)
actions (last_action)	21
joint_vel_rel	21
feet_body_forces	12 (两个足，6D each)
actions (last_action)	21

2. `S_t` 说明：
   1.
      - **Height ([base_pos_z])**: 1D — 判断是否跌倒/躯干高度，低于阈值触发恢复或终止；用于调整膝、踝姿态以维持身高和步态高度。
   2. **Linear velocity ([base_lin_vel])**: 3D — 表示前后/左右/垂直速度，用于控制前进速度、步频和推进力（比如加速、减速或保持巡航速度）。
   3. **Angular velocity / orientation ([base_ang_vel], [base_yaw_roll]), [base_angle_to_target])**: 角速度与朝向量 — 用于稳定姿态（抗摆动）、转向控制和朝向目标调整，决定转向幅度与步态对称性。
   4. **Joint positions（归一化）([joint_pos_limit_normalized])**: N 维（关节数，例中 21）— 表征当前关节配置，决策时用于规划下一步的目标关节角以实现步幅和避免关节极限。
   5. **Joint velocities ([joint_vel_rel])**: N 维 — 反映关节运动趋势，帮助做阻尼、预测下一时刻位移，避免过冲并计算逆动力矩补偿。

6. **Feet forces / wrench ([body_incoming_wrench] for feet)**: 6D×脚数 — 检测接触、着力分布和滑动/不平衡；用于触地时间判断、负载重分配和调整步宽/落点以保持平衡。
7. **Last action ([last_action])**: N 维（动作历史）— 提供历史命令上下文，便于平滑控制（避免突变）、估计执行延迟并做动作修正或滤波。

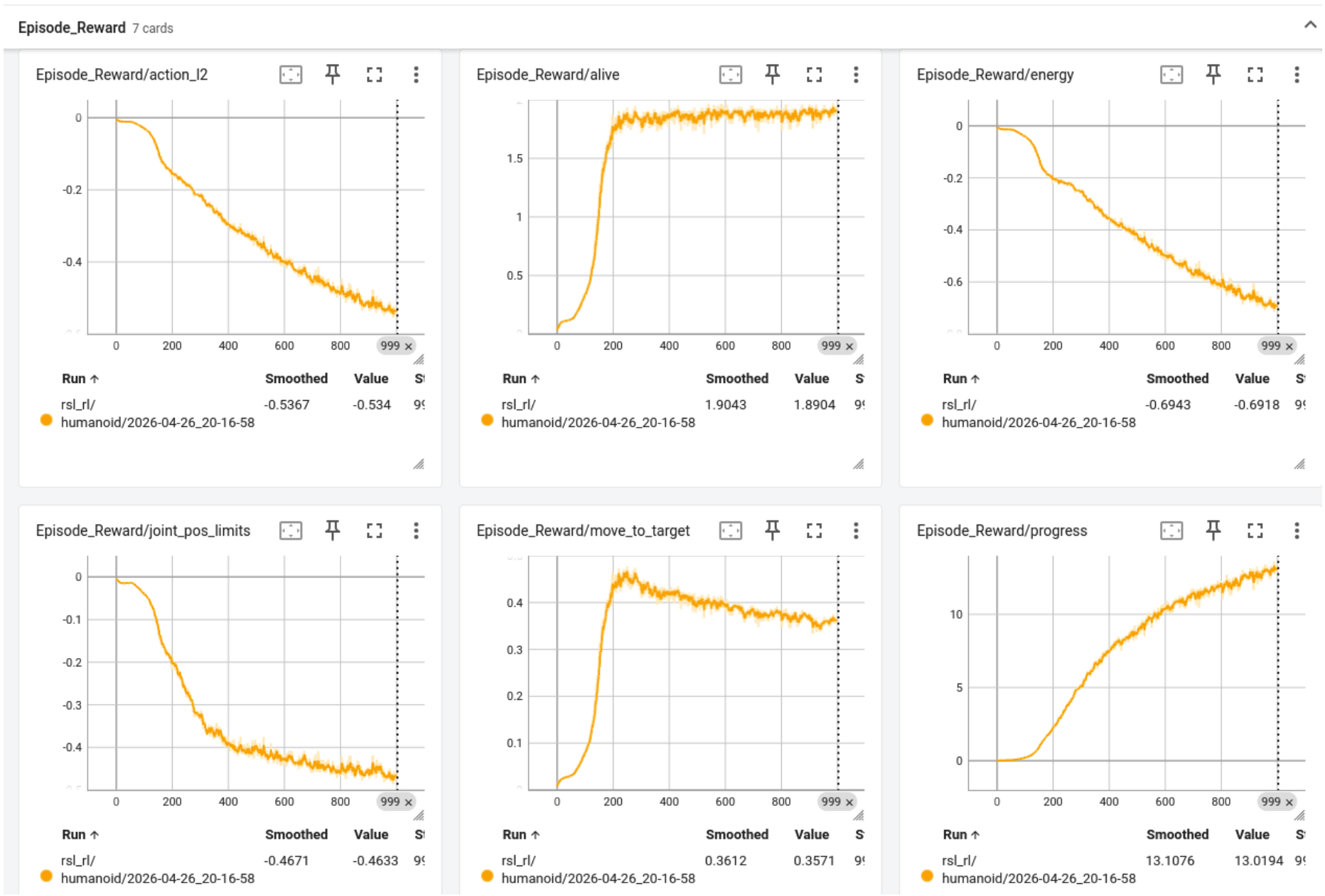
3. 上一帧动作进入观测的主要原因：
   - **提升可观测性（Partial observability）**：上一帧动作补充了环境状态的缺失信息（例如命令未即时产生的物理响应），使策略在部分观测下也能近似马尔可夫决策，从而做出更可靠的选择。
   - **补偿执行延迟 / 动力学迟滞**：真实机器人/仿真存在执行延迟或摩擦滞后，输入上一次动作可以帮助策略推断当前动作尚未完全生效，应如何修正下一步命令来避免过冲或延迟错误。
   - **平滑与稳定控制（动作连续性）**：把历史动作作为输入能让网络学习输出平滑的动作序列（避免突变），在步态控制中减少抖动、提高步态稳定性。
   - **价值估计与方差降低**：动作信息有助于 Critic 更准确地估计下一步返回（因为奖励/下一状态部分依赖于刚执行的命令），从而降低策略梯度的方差，收敛更稳定。
   - **学习残差/差分控制（delta actions）**：网络可以以“上一次动作 + 输出增量”的形式学习控制策略（更易学习微调而非绝对命令）；对步幅微调与姿态修正尤其有用。
   - **检测命令 - 感知不一致（故障/滑移提示）**：若上一帧下发的动作与接触力/速度反馈不一致，策略可据此判断滑移或失稳并触发补偿（例如改变落脚点或增大步幅）。
   - **兼容低频策略 / decimation**：当策略以较低频率输出而仿真步进更细（config 中有 decimation），保留上一帧动作能帮助跨多个模拟步保持控制连续性。
4. 奖励函数中 reward 项说明：
   - **progress**: 奖励机器人沿指定方向前进（正向位移）；鼓励更快移动到目标位置
   - **alive**: 奖励保持存活（不跌倒、未终止）；惩罚跌倒/失败，促使维持平衡和安全姿态。
   - **upright**: 奖励保持躯干直立（姿态接近垂直）；鼓励直立稳定，减少前倾/后仰。
   - **move_to_target**: 奖励朝向并接近目标方向（朝向正确且靠近目标时给分）；促进朝向控制与导航行为。
   - **action_l2**: 对动作幅度做 L2 惩罚（penalize large commands）；惩罚过大/剧烈的动作，鼓励平滑、节能的控制输出。
   - **energy**: 惩罚能量消耗（基于功率/扭矩等计算）；鼓励能量高效的步态与较小力矩使用。
   - **joint_pos_limits**: 惩罚关节接近或超过软极限；避免关节撞限、保护机械约束并促使安全可行的动作范围。
5. 列出所有 termination 条件，并说明它们为什么重要：
   1. **[time_out]**：当当前 episode 长度 >= 最大允许长度时终止。
      - 作用：界定试验时长，防止无效或长期无进展的轨迹占用训练资源；便于稳定批次统计与并行训练。
   2. **[root_height_below_minimum]**（例如 [torso_height]）：当根（躯干）高度低于阈值（如 0.8 m）时终止。
      - 作用：检测跌倒/严重下塌，及时终止失败样本，避免把不可恢复状态当作有价值的数据用于学习，同时提高样本质量与训练效率。
   3. **[bad_orientation]**：当机体朝向/姿态偏离设定角度太多（重力投影角度超过阈值）时终止。
      - 作用：捕捉明显翻倒或极端姿态，保护训练稳定性并促使策略学习维持正常朝向与姿态控制。
   4. **[joint_pos_out_of_limit] / [joint_pos_out_of_manual_limit]**：关节位置超出软极限或手动指定边界时终止。
      - 作用：防止关节撞限或进入不可行的姿态（机械上/物理上不允许），保证训练过程中动作合法且可执行，保护仿真与真实硬件。
   5. **[joint_vel_out_of_limit] / [joint_vel_out_of_manual_limit]**：关节速度超出限制时终止。
      - 作用：避免极端高速动作导致不稳定或数值问题；促进学习平滑、受控的关节速度配置。
   6. **[joint_effort_out_of_limit]**：计算的关节力矩被裁剪（超过 actuator 限制）时终止。
      - 作用：检测控制器输出不可实现（需要超过能力），提示策略产生了不切实际或高风险动作，避免训练无效/危险命令。
   7. **[illegal_contact]**：接触传感器检测到超过阈值的非法接触力时终止。
      - 作用：用于检测撞击、强烈碰撞或传感器指定部位的异常力（例如机器人触碰不该触碰的物体），有助于学习更安全的落脚/避障行为。
   8. **[command_resample]**：基于命令被重采样次数来终止（用于可变长度命令场景）。
      - 作用：在使用命令/任务生成器的设置中控制 episode 长度与样本终止条件，使训练能根据命令序列结构自适应结束。
      为什么这些终止条件重要（总体）：
   - 安全与物理合理性：避免不可恢复或损伤性的状态进入训练数据。
   - 提高样本质量：及时丢弃失败/异常轨迹，使训练更专注于可学习行为。
   - 稳定训练：避免数值发散、极端动作污染策略梯度。
   - 提供明确失败信号：帮助算法快速区分成功/失败、加速学习收敛。
6. 为什么机器人 RL 训练需要并行仿真：
   1. 并行仿真能显著提高样本采集速度与多样性：通过同时运行大量独立环境，训练器能在同一时间内获得更多不同初始状态和交互轨迹，加速策略收敛并降低方差；对基于时间相关数据的 RL（尤其是 on-policy 算法）还能减少样本相关性、稳定梯度估计；此外并行化更高效利用 CPU/GPU 资源、方便做域随机化与鲁棒性测试，并允许更快迭代超参与策略，从而在可接受的墙钟时间内得到高质量的控制策略。
7. 用一段话解释：`Isaac Sim` 和 `Isaac Lab` 的分工分别是什么：

1. Isaac Sim 是 NVIDIA 提供的底层仿真平台（基于 Omniverse），负责高保真物理引擎、渲染、传感器接口、场景与资产管理、并行仿真能力等“仿真基础设施”；Isaac Lab 则是在 Isaac Sim 之上构建的高层工具集与任务库，提供封装好的 RL 环境/MDP（如 `Isaac-Humanoid-v0`）、观测/动作/奖励/终止定义、训练与评估脚本、配置管理、实验复现工具以及与常见 RL 框架的适配，从而把仿真能力转化为可直接用于强化学习研究与工程的流水线。简言之：Isaac Sim 做「仿真引擎和资产」，Isaac Lab 做「任务、框架和训练/实验工具」。

1.4 TensorBoard 查看训练过程

> 1. 打开 TensorBoard：
> `tensorboard --logdir logs`
> 2. 任选你本次训练对应的 run，截图至少 3 条曲线；
> 3. 对每条曲线分别做简短解释，回答：
>
> - 这条曲线反映的是什么量?
> - 它上升 / 下降通常意味着什么?
> - 从你当前这次训练的早期现象看，它说明机器人处于什么阶段?

1. tensorboard 运行截图：



1.

2. 曲线的简短解释：
    1. `Episode_Reward->action_l2`：
        1. 含义：每步动作向量的 L2 平方和；
        2. Episode_Reward/action_l2 趋向 0（数值变大，从负值靠近 0），说明动作幅度在变小，策略变得更平滑/节能（惩罚减小）。若该项变得更负（下降），说明动作的 L2 能量增大，策略输出更大/更激进的力矩或命令（惩罚增大），可能用于快速加速、纠正或探索。总结：上升→更小动作/更节能；下降→更大动作/更激进或更不平滑。
        3. 早期采样（steps 0→19）：Episode_Reward/action_l2 从约 -0.00363 下降到约 -0.01053（绝对值增大），同时 Train/mean_reward 从 ~0.96 上升到 ~1.86，Train/mean_episode_length 也在上升（约 31 → 51）。代理在训练初期增加了动作幅度以产生更多推进力（因此 progress/reward 与 episode length 提升），也因此动作惩罚增大。换言之，当前处于“积极探索 / 学会用更大力矩启动前进”的阶段——代理能让机器人“开始走、活得更久”，但动作还不够节能或平滑。后期通常会看到在保持或提升进度的同时 action_l2 回落（策略学会更高效的动作）。
    2. `Episode_Termination->action_l2`：
        1. 含义：表示在一个记录窗口内因“达到最大 episode 长度（time-out）”而终止的比例/频率（即多少比例的环境在该段时间里是被 max_episode_length 强制结束的），不是单个环境的布尔值。max_episode_length

由 [HumanoidEnvCfg.episode_length_s] 和步长 dt（这里 sim.dt=1/120，decimation=2 → step_dt=1/60）确定，例：16s 对应约 16*60 = 960 步。

2. 上升：更多 episode 因到达最大时长而结束 → 代理更常“活到头”（生存期变长），通常是性能变好（少跌倒、能坚持到完整试验时长）。下降：因 time-out 结束的 episode 比例减少 → 要嘛更多 episode 在更早时刻被其他终止条件（如跌倒、非法接触、越界等）结束（这通常表示更差的生存能力），要嘛只是记录窗口/采样差异；因此要结合 `mean_episode_length` / 其他终止项一起解读。
3. 我们看到 Episode_Termination/time_out 在早期几步非常接近 0（几乎没有时间到期结束），而同时 Train/mean_episode_length 从约 31 上升到 ~50，Train/mean_reward 也在上升。结论：代理正在学习使 episode 更长、获得更高回报，但仍远低于最大时长（≈960 步），所以绝大多数终止是“提前结束”（如跌倒或其它失败），而不是 time-out。换言之，当前处于“早期—正在学习并逐步稳定但尚未能稳健地活到完整 episode 长度”的阶段

3. `Loss->entropy`：
   1. 含义： 表示策略（policy）输出分布的熵（平均信息熵），即动作分布的不确定性或随机性大小；对连续高斯策略通常由 log-std/std 控制。
   2. 上升/下降意味着什么 — 熵上升：策略更随机、探索性更强；熵下降：策略更确定、收敛到确定性动作（探索减少）。在训练中熵通常被当作正则项来平衡探索与收敛（高熵促进探索，低熵有助于利用）。
   3. 当前这次训练的早期现象说明（基于日志样本） — 早期 `Loss/entropy` 从约 29.75 逐步下降到 ~26.6（持续下降趋势），说明训练初期策略渐渐减少随机性、收敛到更确定性的动作；结合我们看到的 mean_reward 和 mean_episode_length 同步上升，这表明代理正从探索阶段过渡到学会稳定主动前进的阶段：它开始利用已学到的行为（减少噪声）以提高回报，同时动作幅度（action_l2）短期内上升以生成推进力，随后应会再降低以变得更高效。

基础作业提交要求

请将以上内容整理为 **一份 PDF**，包含必要的截图说明、步骤说明、问题的回答。